

FPGA-based Neuromorphic Acceleration of Spiking Neural Networks for Event-Driven Humanoid Robotics

Aruki Komatsuzaki ^{1, *}, Atharv Sharma ¹, Yuga Hanyu ¹, Khanh N. Dang ¹, Abderazek Ben Abdallah ¹ and Zhishang Wang ¹

¹ The School of Computer Science and Engineering, The University of Aizu, Aizu-Wakamatsu, Fukushima, Japan;

s1310244@u-aizu.ac.jp

m5292015@u-aizu.ac.jp

m5291018@u-aizu.ac.jp

khanh@u-aizu.ac.jp

benab@u-aizu.ac.jp

zwang@u-aizu.ac.jp

* Correspondence: s1310244@u-aizu.ac.jp

Abstract

The deployment of modern AI in embedded humanoid robotics is increasingly hindered by high computational latency and energy demands. Neuromorphic computing, through event-driven processing and spike-based information representation, provides a compelling alternative for low-power intelligence. This work proposes a high-performance neuromorphic accelerator for Spiking Neural Network (SNN) based GAN generator, implemented on the AMD VCK5000 Versal FPGA platform. By leveraging a hardware-software co-design methodology, the architecture exploits spike sparsity and reconfigurable logic to eliminate the need for power-hungry DSP blocks. This zero-DSP utilization significantly reduces the hardware power footprint while overcoming the bottlenecks of traditional multiply-accumulate (MAC) operations. Experimental results demonstrate that the proposed SNN generator implementation achieves a $1.86 - 3.86 \times$ execution speedup over conventional ANN generator baseline. The system-level analysis confirms that this logic-centric approach enables real-time, energy-efficient inference, making it ideal for the stringent constraints of autonomous robotic platforms.

Keywords: Neuromorphic Computing; Spiking Neural Networks; FPGA Acceleration; Event-Driven Computing; Humanoid Robotics.

1. Introduction

Generative Adversarial Networks (GANs) are a foundational technology in generative AI, achieving strong performance in content generation. In GANs, two learnable models, the discriminator and the generator, compete with one another to synthesize new data that mimics original patterns [1, 2]. Although this learning process yields highly realistic outputs, GANs often consist of complex models that demand high computational capability [3]. Consequently, their high computational cost significantly limits deployment on resource-constrained devices, such as embedded humanoid robotic systems [4]. These robotic systems require real-time, low-latency processing within strict power

Academic Editor: Aruki Komatsuzaki

Published: date

Copyright: © 2026 by the authors. Submitted for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

budgets, making traditional Artificial Neural Networks (ANNs) challenging to implement efficiently.

To reduce computational complexity and realize energy-efficient AI systems, neuromorphic computing offers a highly promising alternative [5, 9]. Spiking Neural Networks (SNNs) provide simpler computation by using action potentials (spikes) as the primary elements for processing [6]. By leveraging event-driven and multiplication-free computation, SNNs can significantly reduce computational cost while maintaining comparable performance to traditional ANNs.

While SNNs offer theoretical efficiency, realizing their full potential requires highly optimized physical hardware implementations [7]. Deploying complex neural network models on Field-Programmable Gate Arrays (FPGAs) presents specific design challenges due to strict resource constraints, including limited Look-Up Tables (LUTs), Flip-Flops (FFs), and Block RAMs (BRAMs) [8]. Therefore, mapping the operations of an SNN onto programmable logic requires a dedicated neuromorphic architecture that exploits spike sparsity and memory efficiency to enable real-time inference without relying on power-hungry computational blocks [9].

In order to address these challenges and expand the feasibility of generative AI in robotic applications, this work presents the physical hardware implementation of an SNN-based classification and generative model on the AMD VCK5000 Versal FPGA platform [12, 13]. Building upon the foundational concepts of converting an ANN generator into a mathematically equivalent SNN [10], we propose a comprehensive hardware-software co-design framework. This work demonstrates approach to deploying spike-based generative models specifically targeted for event-driven humanoid robotics. The major contributions of this paper are as follows:

- An SNN-based framework using ANN-to-SNN conversion optimized for both classification and generative tasks.
- A hardware-software co-design methodology for mapping event-driven SNN operations onto the AMD VCK5000 Versal FPGA.
- A comprehensive evaluation of system-level behavior, execution flow, hardware resource utilization, and on-chip power breakdown for a $100 \times 1024 \times 1024 \times 784$ network topology.

The remainder of this paper is organized as follows. Section 2 presents the proposed framework, including the software conversion methodology and hardware architecture. Section 3 describes the experimental setup, RTL simulation, and FPGA evaluation results. Finally, Section 4 concludes the paper.

2. Proposed Framework

The architecture of the proposed SNN-GAN framework consists of two primary domains: software modeling and hardware implementation. Fig. 1 summarizes the design flow of the proposed framework for GAN model.

The design flow begins in the software domain, where an ANN-based GAN is trained, and subsequently converted into its SNN counterpart. Following model validation, critical network parameters; specifically, weights, biases, and input spikes are extracted for hardware mapping. In the hardware domain, these parameters are integrated into on-chip memory modules to facilitate the instantiation of the SNN-GAN within the programmable logic. The hardware development flow proceeds through RTL simulation for logic validation, followed by FPGA synthesis and final device implementation.

This integrated hardware-software co-design strategy ensures that the event-driven characteristics of the spiking model are effectively translated into optimized hardware-level architectural efficiencies.

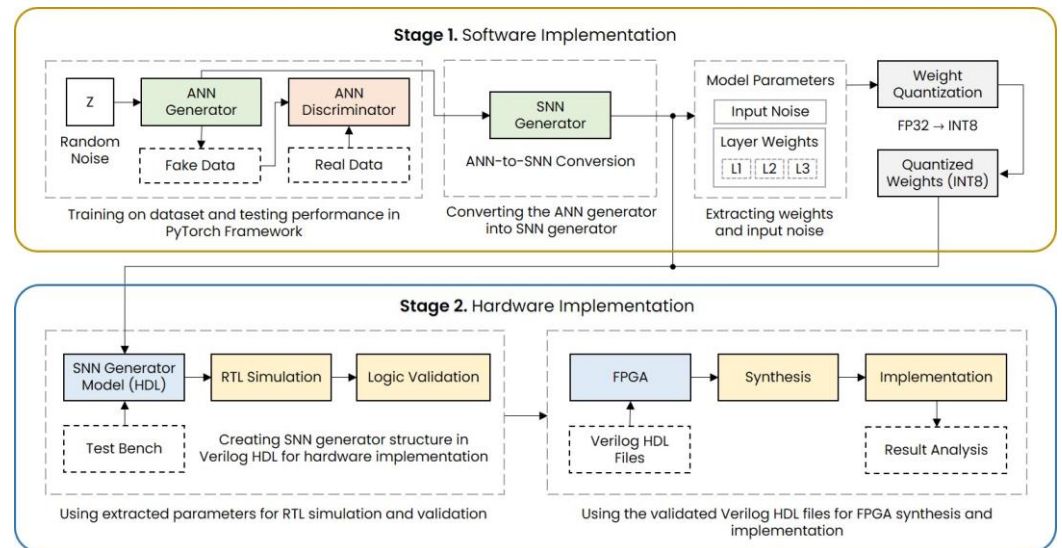


Figure 1. Design flow of the implementation framework for GAN model.

2.1. Software

In the software phase, the primary objective is to train the generative model and translate it into a hardware-friendly spiking architecture. The training process follows standard adversarial training techniques. The discrete empirical objective function of the GAN for a mini-batch of size m is defined as a minimax game between the discriminator (D) and the generator (G):

$$\min_G \max_D V(D, G) = \frac{1}{m} \sum_{n=1}^m [\log D(X_n) + \log(1 - D(G(Z_n)))] \quad (1)$$

where X_n represents the real data and Z_n is the input noise. The generator minimizes this value to fool the discriminator, while the discriminator maximizes it.

Once the ANN is successfully trained and the desired output quality is achieved, the generator is converted into an equivalent SNN using ANN-to-SNN conversion [10]. This conversion is accomplished by mapping the ReLU activation functions of the ANN to the firing rates of Leaky Integrate-and-Fire (LIF) neurons. The LIF neuron model is selected due to its operational simplicity; it accumulates membrane potential and generates a spike upon reaching a specific threshold. This event-driven mechanism effectively eliminates the need for multiplication operations, thereby significantly reducing hardware area cost and energy consumption.

Following the ANN-to-SNN conversion, the converted network is validated by comparing the reconstructed images generated by the SNN with those from the original ANN. Finally, the exact synaptic weights, neuron biases, and input spike trains are extracted to serve as the foundational data for the hardware implementation.

2.2. Hardware

The hardware architecture is optimized to execute spike-based operations efficiently within the programmable logic of the FPGA. As illustrated in Fig. 2, the system is organized hierarchically to support large-scale generative models. The fundamental computational unit is the computing core shown in Fig. 2(b), which integrates a synapse crossbar for weighted accumulations and a Leaky Integrate-and-Fire (LIF) neuron array for spike generation. These cores are tiled to form high-performance neuromorphic chips shown in Fig. 2(c), which are designed to handle the high-dimensional data of the SNN-GAN topology, shown in Fig. 2(a). This hardware acceleration provides the low-latency processing necessary for real-time humanoid robotic applications, as depicted in Fig. 2(d).

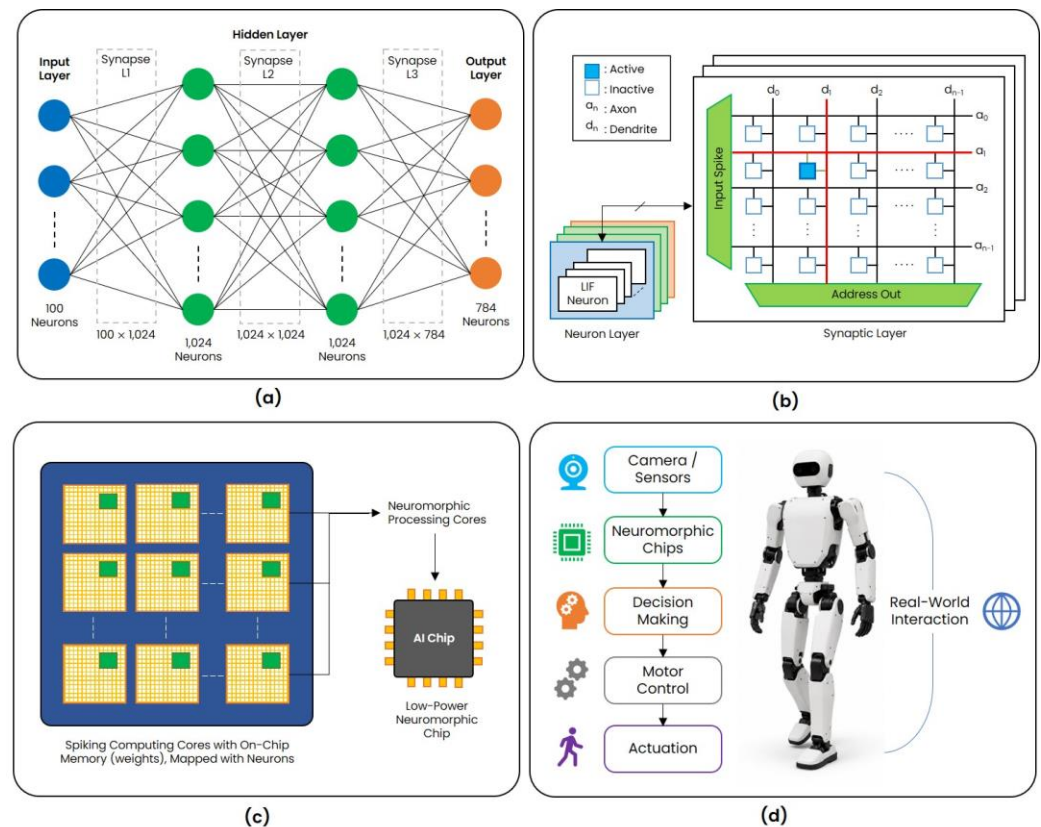


Figure 2. Hardware architecture of the SNN-GAN generator. (a) Structure of GAN Spiking Neural Network. (b) Computing core; consisting of synapse crossbar and neuron layer. (c) Neuromorphic computing chips with computing core. (d) Humanoid robot application using neuromorphic chips.

To store the synaptic weights and biases extracted from the software domain, we utilize OpenRAM-based SRAM macros [11]. SRAM macro is of size 1024×8 , consisting of 1024 memory words with word length of 8-bits, configured to accommodate the $100 \times 1024 \times 1024 \times 784$ network topology for architectural design and RTL simulation. These extracted parameters are embedded inside the weights, biases, and spike ROM modules for RTL simulation and subsequent FPGA implementation.

The hardware operates in a timestep-driven manner, processing input spikes as they propagate through multiple synaptic layers sequentially. Within each layer, the synapse crossbar performs weighted accumulations, which are then passed to the LIF neuron array for membrane potential updates and spike generation. Because these neuron states are updated only upon the arrival of a spike, the architecture effectively exploits spike sparsity to minimize unnecessary memory accesses and computational overhead.

The hardware validation flow consists of two primary stages. First, RTL simulation is conducted to perform logic validation and ensure functional correctness across the synaptic layers. Once the logical equivalence between the software model and hardware logic is verified, the design proceeds to FPGA synthesis and final physical device implementation.

3. Evaluation

3.1. Experimental Setup

To evaluate the designed SNN-based GAN generator architecture, the software and hardware environments were configured with specific parameters. Table 1 details the datasets, network topology, and data formats used during both the software training and hardware execution phases.

At the software level, the original ANN-based GAN model and its corresponding SNN-based GAN implementation were developed in Python using the PyTorch framework, with all computations performed in FP32 precision. Model training and evaluation were conducted on a high-performance workstation. To facilitate subsequent hardware implementation, the trained synaptic weights and biases were quantized from FP32 to INT8 precision while preserving the original network architecture and inference behavior.

Table 1. Model architecture and parameters

Parameter	Value
Training Data	MNIST Handwritten Digits
Training Numbers	600 Images per Batch × 20 Epochs
Input Layer	100 Neurons
Layer 1	1,024 Neurons
Layer 2	1,024 Neurons
Layer 3 (Output)	784 Neurons
Software Model Parameter Format	Signed FP32
RTL Model Parameter Format	Signed INT8

For hardware-level evaluation, the system was synthesized and deployed on the AMD VCK5000 Versal FPGA platform [12, 13]. This platform provides a representative environment for profiling hardware resource utilization and measuring power consumption under realistic operating conditions. Table 2 summarizes the detailed specifications of the hardware and software environments.

Table 2. Hardware and software environment setup

Component	Specification
Software (Host PC)	Workstation (raytrek 4CZ46+)
OS	Windows 11
Framework	PyTorch (Python)
CPU	Intel Core i9-14900KF
RAM	128GB (32GB × 4)
GPU	NVIDIA GeForce RTX 4060 8GB
Storage	4TB SSD (NVMe Gen4)
Hardware (FPGA)	AMD VCK5000 Versal Development Card
Architecture	Versal 7nm ACAP
LUTs / FFs	~ 899,840 / ~ 1,799,680
DSP Engines	1,968

3.2. Computation Comparison

A comparative analysis was performed to evaluate the computational efficiency of the original ANN generator against its converted SNN counterpart. Table 3 summarizes the total number of linear layer function calls and the cumulative GPU execution time required by both models across various image scales.

While the SNN operates over multiple timesteps, its event-driven nature and multiplication-free accumulation operations significantly reduce the overall computational overhead. Consequently, the SNN is observed to achieve a lower execution time while maintaining generation fidelity comparable to the ANN baseline. This reduction in execution time highlights the intrinsic efficiency of spike-based processing, which minimizes redundant computations through temporal sparsity.

Table 3. Computational comparison between ANN and SNN generator

Images	Generator	Linear Function Calls	Total GPU Time (ms)
100	ANN	300	72.94
	SNN	10,500	18.90
200	ANN	600	74.71
	SNN	21,000	39.61
400	ANN	1,200	69.24
	SNN	42,000	22.71
600	ANN	1,800	69.55
	SNN	63,000	27.70
800	ANN	2,400	69.24
	SNN	84,000	33.50
1,000	ANN	3,000	73.60
	SNN	105,000	39.552

The computational performance of the ANN and SNN generators is analyzed in Fig. 3, focusing on execution time and functional complexity across a scaling range of 100 to 1,000 images.

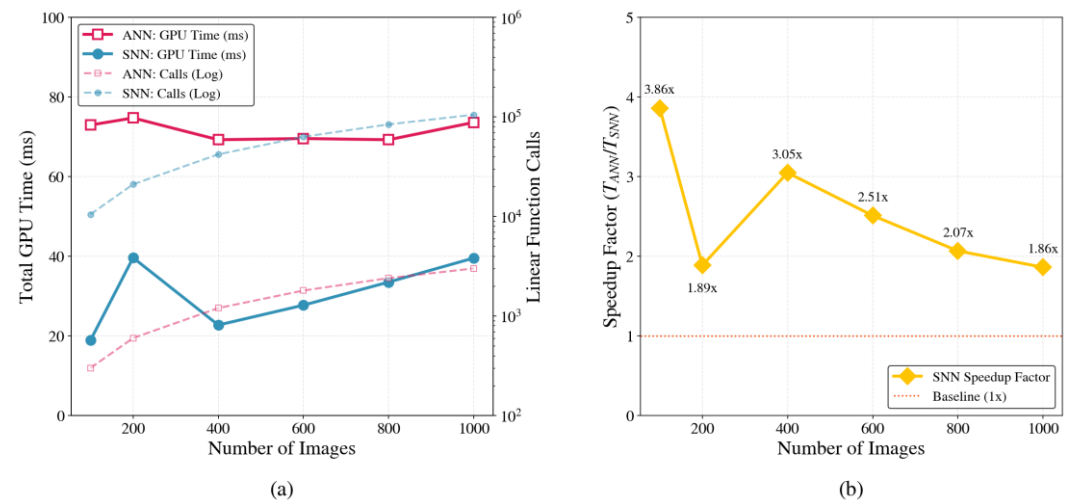


Figure 3. Performance analysis of ANN and SNN generators: (a) Total GPU execution time and linear function calls across different image scales. (b) SNN speedup ratio relative to the ANN.

Notably, the SNN achieves lower execution times despite requiring more function calls due to its multi-timestep nature. The resulting $1.86 - 3.86 \times$ speedup demonstrates the efficiency of event-driven computation in reducing the total GPU workload.

3.3. RTL Logic Validation

Before proceeding to physical hardware implementation, the behavior of the SNN-based GAN model was validated through RTL simulation. The exact synaptic weights, biases, and input spike trains extracted during software implementation were loaded into the ROM modules to simulate the hardware logic.

3.4. FPGA Implementation

The SNN-GAN model was synthesized and implemented on the AMD VCK5000 Versal FPGA platform to demonstrate its feasibility for resource-constrained embedded systems.

The comparative hardware footprints are illustrated in Fig. 4. The SNN-GAN generator (Fig. 4a) requires a heavily dense implementation, spanning the majority of the FPGA fabric to accommodate its large-scale $100 \times 1024 \times 1024 \times 784$ architecture. In contrast, the SNN-MLP (Fig. 4b) of structure $784 \times 48 \times 10$ occupies a minimal footprint, utilizing only a small localized region. This visual comparison highlights the significantly higher complexity and resource demand of generative spiking architectures.

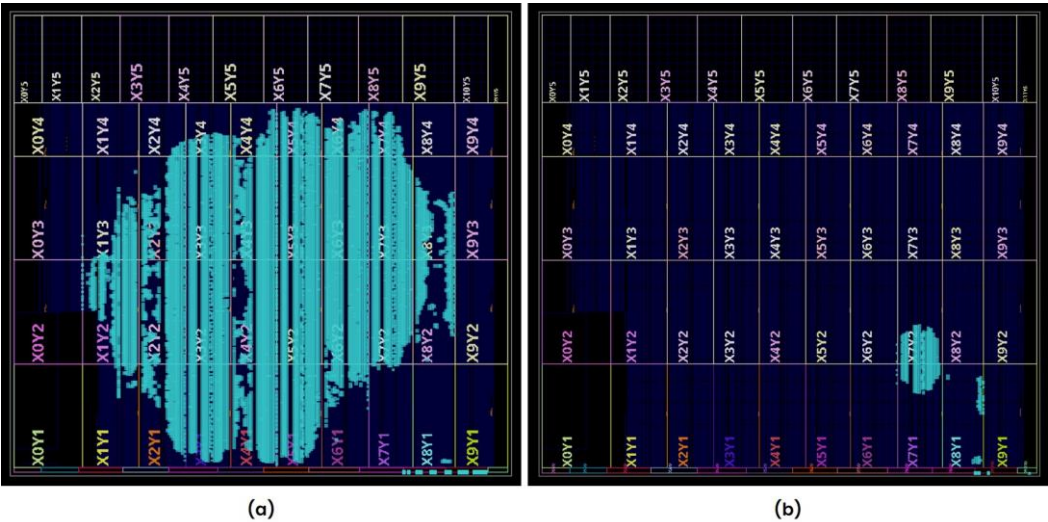


Figure 4. FPGA floorplan comparison highlighting hardware scale and resource density: (a) SNN-GAN generator. (b) SNN-MLP classification model.

Table 4 summarizes the FPGA resource utilization, including Look-Up Tables (LUTs), Flip-Flops (FFs), and Block RAMs (BRAMs). Notably, the spike-driven nature of the computation aims to minimize or completely eliminate the use of Digital Signal Processing (DSP) slices. Table 5 provides the on-chip power consumption report, broken down by dynamic and static power components.

Table 4. FPGA resource utilization

Resource	Used	Available	Utilization (%)
LUT	307,977	899,840	34.22
LUTRAM	10,528	449,920	2.34
FF	81,093	1,799,680	4.40
BRAM	580	967	59.98
DSP	0	1,968	0.00

Table 5. FPGA on-chip power breakdown

Category	Component	Power (W)	Share (%)
Dynamic Power	Clocks	0.475	15
	Signals	1.007	33
	Logic	1.150	37
	BRAM	0.461	15
	Dynamic Total	3.095	19
Static Power	Device Static	13.496	81
Total On-Chip Power		16.591	100

4. Conclusion

This work successfully demonstrated the hardware acceleration of a large-scale generative Spiking Neural Network (SNN-GAN) on the AMD VCK5000 Versal platform. By leveraging a hardware–software co-design methodology, the proposed architecture achieves a complete elimination of power-hungry DSP blocks, performing all spike-based computations entirely within reconfigurable logic. This DSP-free approach utilizes 34.22% of available LUTs and 59.98% of on-chip BRAM, effectively shifting the computational burden away from traditional multiply-accumulate (MAC) units to highly efficient, event-driven logical operations.

The implementation exhibits superior energy efficiency, with dynamic power consumption restricted to just 3.095 W, representing only 19% of the total on-chip power. Furthermore, the event-driven nature of the accelerator provides a significant performance advantage, delivering a 1.86–3.86x speedup over conventional ANN baselines. These results confirm that logic-centric, neuromorphic architectures can provide the low-latency and low-power generative capabilities essential for the real-time cognitive demands of embedded humanoid robotic systems. Future work will focus on the physical deployment of this neuromorphic architecture into embedded humanoid robotic systems to evaluate real-world, system-level performance.

References

1. Goodfellow, Ian, et al. "Generative adversarial networks." *Communications of the ACM* 63.11 (2020): 139-144.
2. Aggarwal, Alankrita, Mamta Mittal, and Gopi Battineni. "Generative adversarial network: An overview of theory and applications." *International Journal of Information Management Data Insights* 1.1 (2021): 100004.
3. Wang, Kunfeng, et al. "Generative adversarial networks: introduction and outlook." *IEEE/CAA Journal of Automatica Sinica* 4.4 (2017): 588-598.
4. Tong, Yuchuang, Haotian Liu, and Zhengtao Zhang. "Advancements in humanoid robots: A comprehensive review and future prospects." *IEEE/CAA Journal of Automatica Sinica* 11.2 (2024): 301-328.
5. Liu, Dingbang, Hao Yu, and Yang Chai. "Low-power computing with neuromorphic engineering." *Advanced Intelligent Systems* 3.2 (2021): 2000150.
6. Merolla, Paul A., et al. "A million spiking-neuron integrated circuit with a scalable communication network and interface." *Science* 345.6197 (2014): 668-673.
7. Bouvier, Maxence, et al. "Spiking neural networks hardware implementations and challenges: A survey." *ACM Journal on Emerging Technologies in Computing Systems (JETC)* 15.2 (2019): 1-35.
8. Mittal, Sparsh. "A survey of FPGA-based accelerators for convolutional neural networks." *Neural computing and applications* 32.4 (2020): 1109-1139.
9. Rathi, Nitin, et al. "Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware." *ACM Computing Surveys* 55.12 (2023): 1-49.
10. Diehl, Peter U., et al. "Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing." 2015 International joint conference on neural networks (IJCNN). iee, 2015.
11. Guthaus, Matthew R., et al. "OpenRAM: An open-source memory compiler." *Proceedings of the 35th International Conference on Computer-Aided Design*. 2016.
12. Advanced Micro Devices, Inc. (AMD). AMD VCK5000 Versal Development Card Product Brief. Available online: <https://japan.xilinx.com/content/dam/xilinx/publications/product-briefs/amd-xilinx-vck5000-product-brief.pdf>.
13. Advanced Micro Devices, Inc. (AMD). VCK5000 Gen4x8 Vitis Platform User Guide. Available online: https://xilinx.github.io/Alveo-Versal-Platforms/alveoversalplatforms/build/html/docs/user-guide/ug_vck5000_gen4x8_qdma.html.